

Multi-Armed Bandit (MAB)

Genya Ishigaki

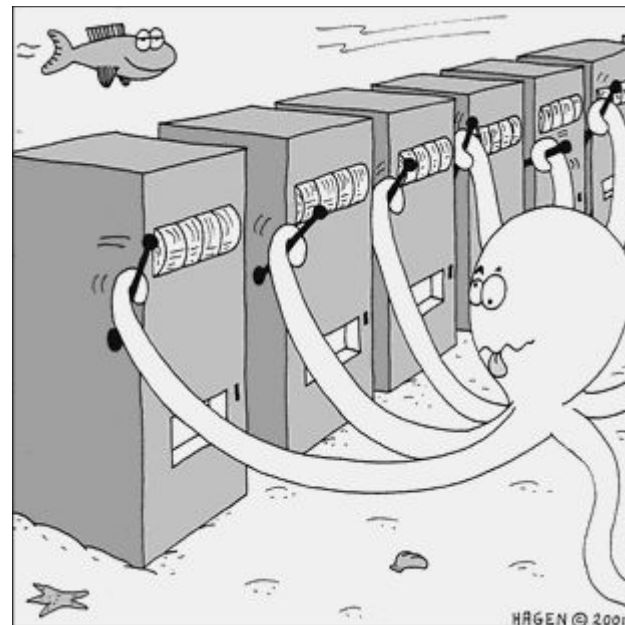
Evaluative feedback vs. Instructive feedback



- What distinguishes reinforcement learning from other types of learning
- The use of training information that evaluates the actions taken rather than *instructs* by giving correct actions
 - **Evaluative feedback** indicates how good the action taken was, but not whether it was the best or the worst action possible
 - **Instructive feedback** indicates the correct action to take, independently of the action actually taken (supervised learning)
- We are going to analyze the fundamental characteristics of evaluative feedback using the Multi-Armed Bandit problem.

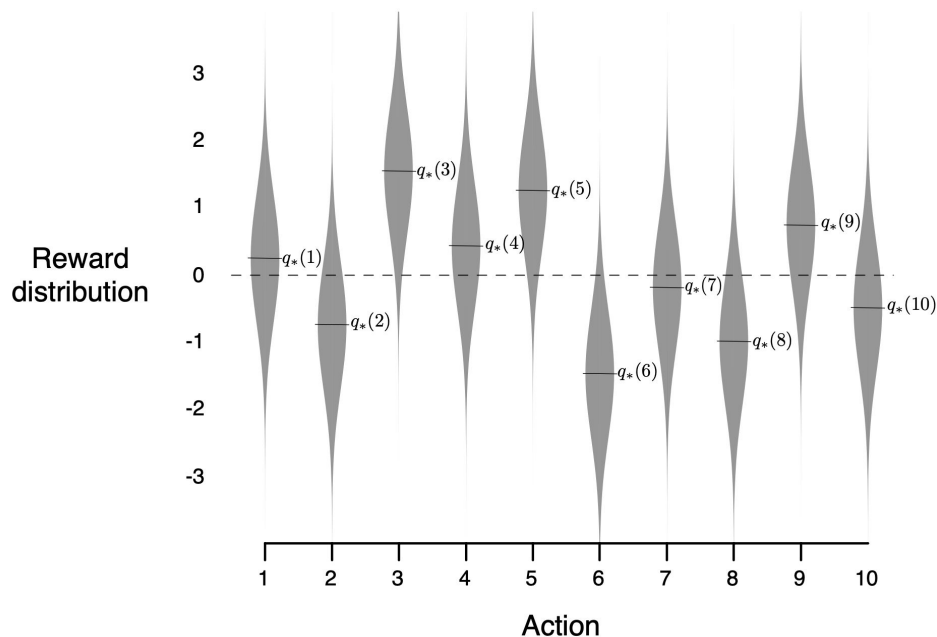
The Bandit

- **Assume:** k slot machines in front of you
- You will select one of the machines to bet at each round
- After each choice, you receive a numerical reward chosen from a stationary probability distribution
- **Goal:** find a best slot machine to bet on average



The Bandit

- If you are given the underlying distribution, you win!
- In practice, the distribution is unavailable
- A key is to find out the true expected reward



The Value of an Action



↑ val only for action

Reward taking action a

- Similar to the state values, we can define the value of action (slot machine)

$$\underline{q_*(a)} \doteq \mathbb{E}[R_t | A_t = a]$$

Basically remove state from function

- Only way to estimate the value is to collect more rewards
- **Exploiting**: Repeat good actions based on your current knowledge of the values of the actions *Repeat good actions, but don't try anything new, so don't know it better options*
- **Exploring**: Try new actions or bad actions to improve your estimate of the actions' values
- How do you balance these two?

ϵ -greedy Method

$\uparrow$ val average performance of action

- Choose a random action with probability ϵ
- Use a greedy action with probability $1 - \epsilon$

a_1

a_2

1	1					100	180	100	110
		2	2	2	4				

Handwritten notes:

- Red arrows pointing to 1 and 1 in row a_1 : $q(a_1) = 1$, $q(a_2) = ?$
- Red arrow pointing to 100 in row a_1 : $q(a_1) = \frac{102}{3}$
- Red arrow pointing to 2 in row a_2 : $q(a_1) = 1$, $q(a_2) = ?$
- Red arrow pointing to 2 in row a_2 : $q(a_1) = 1$, $q(a_2) = 2$

Green arrows indicate transitions between actions.

Assume the initial estimate is 0

A simple bandit algorithm

Initialize, for $a = 1$ to k :

$$Q(a) \leftarrow 0$$

$$N(a) \leftarrow 0 \text{ \# Visits}$$

Loop forever:

$$A \leftarrow \begin{cases} \operatorname{argmax}_a Q(a) & \text{with probability } 1 - \epsilon \\ \text{a random action} & \text{with probability } \epsilon \end{cases} \quad (\text{breaking ties randomly})$$

$$R \leftarrow \text{bandit}(A)$$

$$N(A) \leftarrow N(A) + 1$$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$$

Incremental Implementation



- To compute the average after i -th action, a naive implementation requires a list of all rewards

$$Q_n \doteq \frac{R_1 + R_2 + \cdots + R_{n-1}}{n - 1}$$

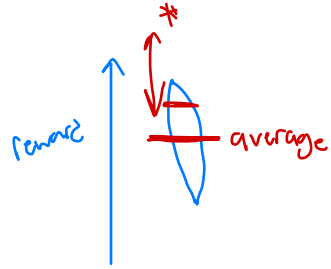
- The memory and computational requirements would grow over time as more rewards are seen

Incremental Implementation



$$\begin{aligned} Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\ &= \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) \\ &= \frac{1}{n} \left(R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i \right) \end{aligned}$$

Incremental Implementation



$$\begin{aligned}Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\&= \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) \\&= \frac{1}{n} \left(R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i \right) \\&= \frac{1}{n} \left(R_n + (n-1) Q_n \right) \text{ This average} \\&= \frac{1}{n} \left(R_n + nQ_n - Q_n \right) \\&= Q_n + \frac{1}{n} [R_n - Q_n],\end{aligned}$$

how fast you want
to update avg

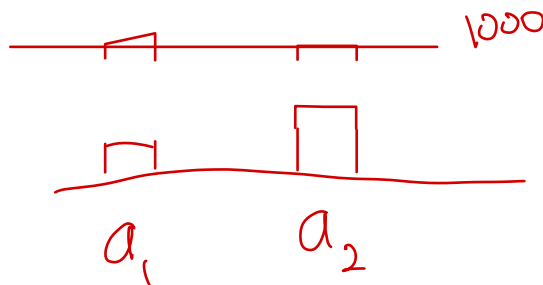
$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \left[\text{Target} - \text{OldEstimate} \right]$$

Optimistic Initial Values

- Exploration depends to the initial action-value estimates (biased by the initial values)
- Initial action values can also be used as a simple way to encourage exploration

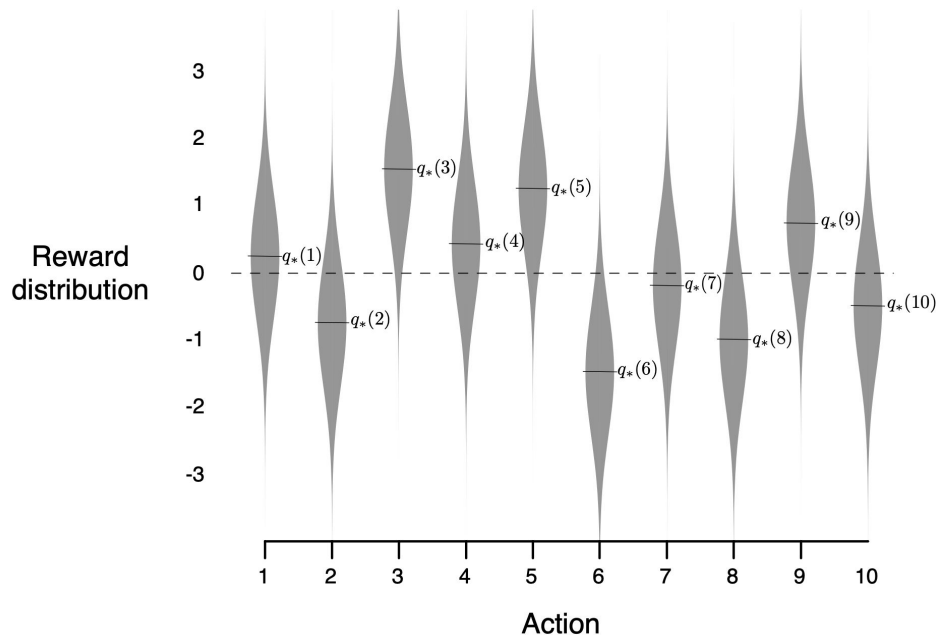
$$a \leftarrow \underset{a}{\operatorname{argmax}} Q(a)$$

$$Q(a) \leftarrow +1000$$



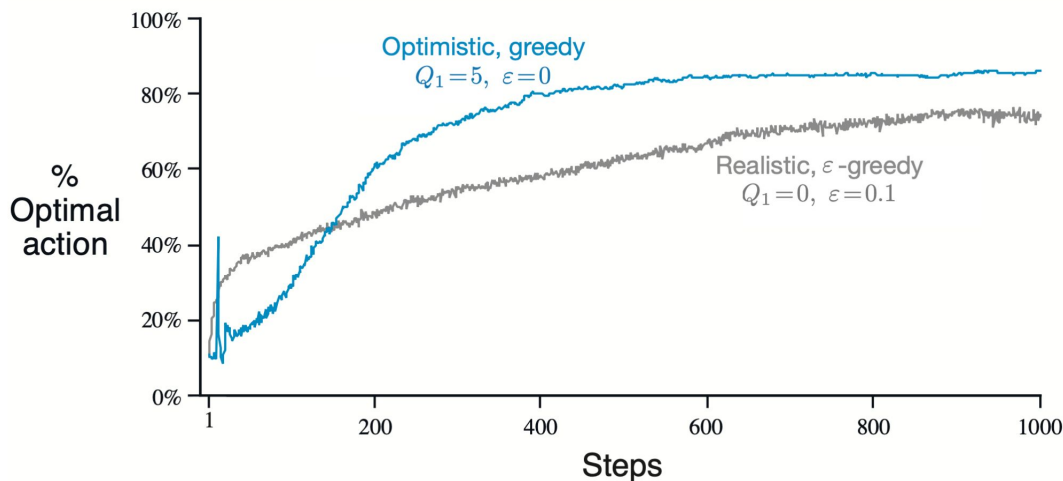
Milestone Questions

- What kind of initialization should we do (to encourage exploration)?
 - Optimistic (much higher than the expected true mean) *shoot for the moon*
 - Realistic (much less than the expected true mean)



Optimistic Initial Values

- e.g. An initial estimate of +5 is wildly optimistic (based on the previous diagram)
- This optimism encourages action-value methods to explore
- The learner switches to other actions, being “disappointed” with the rewards it is receiving



Upper-Confidence-Bound (UCB)



- Is there anyway to quantify the accuracy of the value estimate?
- In a stationary scenario, more samples should give better estimates (Law of Large Numbers)
- We will define the Upper Confidence Bound (UCB) of each arm so that
 - It becomes larger if the arm has not been visited too many times
- Action selection can be greedy based on the current estimate and the UCB

$$a_t = \operatorname{argmax}_{a \in A} \left(\overline{Q}_t(a) + U_t(a) \right)$$

$\uparrow$ $\uparrow$
- 10 ~ 10
 ~ 1

Upper-Confidence-Bound (UCB)



Hoeffding's Inequality

- Let $X_1, X_2, \dots, X_t$ be i.i.d. random variables in $[0, 1]$, and
- let $\overline{X}_t$ be the sample mean. Then,

avg

$$\mathbb{P}\left[\mathbb{E}[X] > \overline{X}_t + u\right] \leq e^{-2tu^2}$$

actual
expectation

mean + u

bounds
sample
mean

Upper-Confidence-Bound (UCB)



Hoeffding's Inequality

- Let $X_1, X_2, \dots, X_t$ be i.i.d. random variables in $[0, 1]$, and
- let $\overline{X_t}$ be the sample mean. Then,

$$\mathbb{P}\left[\mathbb{E}[X] > \overline{X_t} + u\right] \leq e^{-2tu^2}$$

- We want to use the Hoeffding's Inequality to guarantee that our greedy based on the UCB rarely exceed the actual Q-values

$$\mathbb{P}\left[Q(a) > \overline{Q_t}(a) + U_t(a)\right] \leq e^{-2N_t(a)U_t(a)^2} (\doteq p)$$

Upper-Confidence-Bound (UCB)



$$p = e^{-2N_t(a)U_t(a)^2}$$
$$U_t(a) = \sqrt{\frac{-\log p}{N_t(a)}}$$

used for uncertainty
for specific action
at time t

- Reduce the probability p of the UCB value exceeding the actual Q-values over time (e.g. $p = t^{-4}$)

$$U_t(a) = \sqrt{\frac{2 \log t}{N_t(a)}}$$

number time
we try action

Upper-Confidence-Bound (UCB)



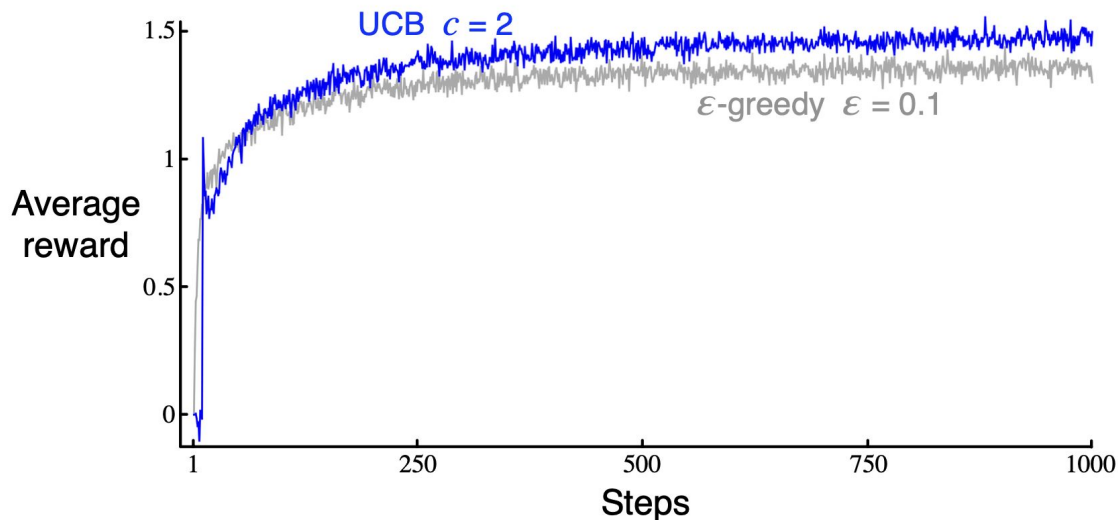
- Quantify the uncertainty about the accuracy of the action-value estimates
- ϵ -greedy action selection forces the non-greedy actions to be tried, but indiscriminately, with no preference for those that are nearly greedy or particularly uncertain

$$a_t = \operatorname{argmax}_{a \in \mathcal{A}} \left(\bar{Q}(a) + \sqrt{\frac{2 \log t}{N_t(a)}} \right)$$

Comparing UCB and Epsilon-greedy

Provides theoretical guarantee

- In practice, the epsilon-greedy works really well! (but may require some tuning)
- UCB is a safe choice as it has theoretical guarantee to perform well in general (not necessarily the best)



Coding



- The epsilon-greedy
- UCB
- (Optimistic initialization)